



Advanced Web Programming

Ung Văn Giàu
Email: giau.ung@eiu.edu.vn



Interacting with RESTful APIs

Topic

We'll cover the following topics:

- Getting set up
- Using the effect hook with fetch
- Posting data with fetch
- Using React Router

Technical requirements

- Node.js and npm
- Visual Studio Code



Creating the project

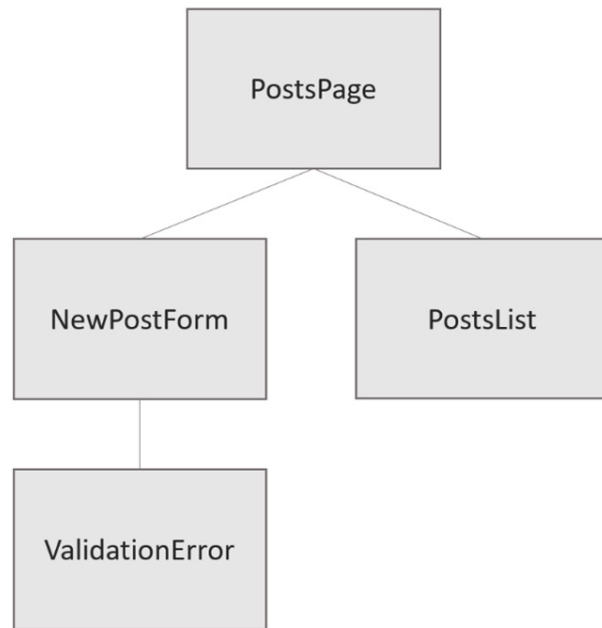
Setting up the project

- Create a React and TypeScript project
- Install and configure Tailwind
- Install React Router
- Install React Hook Form

Creating the project

Understanding the component structure

- The app will be a single page containing a form for adding new posts above a list of all the existing posts.
- The app will be structured into the following components:
 - ✓ **PostsPage** will render the whole page by referencing the **NewPostForm** and **PostsList** components. It will also interact with the REST API.
 - ✓ **NewPostForm** will render a form that allows a user to enter a new blog post. This will use the **ValidationError** component to render validation error messages.
 - ✓ **PostsList** will render the list of blog posts.



Creating the project

Creating a REST API

- Create a REST API using a tool called **JSON Server**, which allows a REST API to be quickly created.
- Install JSON Server by running the following command:

```
npm i -D json-server
```

Creating the project

Creating a REST API

- We define the data behind the API in a JSON file.
- Create a file called **db.json** in **the root of the project** containing the following:

```
{
  "posts": [
    {
      "title": "Getting started with fetch",
      "description": "How to interact with backend APIs using fetch",
      "id": 1
    },
    {
      "title": "Getting started with useEffect",
      "description": "How to use React's useEffect hook for interacting with backend APIs",
      "id": 2
    }
  ]
}
```


Creating the project

Creating a REST API

- We need to **define an npm script** to **start** the JSON server and handle requests.
- Open **package.json** and add a script called server as follows:

```
{  
  ...,  
  "scripts": {  
    ...,  
    "server": "json-server --watch db.json --port 3001"  
  },  
  ...  
}
```

- Start the API by running the script: **npm run server**

Using the effect hook with fetch

Getting blog posts using fetch

- We will start by creating a function that gets blog posts from the REST API using the browser's **fetch** function; we will store the **API URL** in an environment variable.
- Carry out the following steps:
 - Create a file called **.env** in the **root of the project** containing this variable

```
VITE_API_URL = http://localhost:3001/posts/
```

This environment variable is injected into the code at build time and can be accessed by code using **import.meta.env.VITE_API_URL**

Env Variables

- Vite exposes env variables under **import.meta.env** object as strings automatically. These constants are defined as global variables during dev and statically replaced at build time.
- To prevent accidentally leaking env variables to the client, only variables prefixed with **VITE_** are exposed to your Vite-processed code

```
.env file
VITE_SOME_KEY=123
DB_PASSWORD=foobar

# Source code
console.log(import.meta.env.VITE_SOME_KEY) // "123"
console.log(import.meta.env.DB_PASSWORD) // undefined
```

- Vite loads additional environment variables from the following files: `.env`, `.env.local` (ignored by git)

Using the effect hook with fetch

Getting blog posts using fetch

- Carry out the following steps:
 - Create a folder called **posts** in the **src** folder for all the files for the blog post feature.
 - Create a file called **getPosts.ts** in the **posts** folder and add the following function that gets the blog posts:

```
export async function getPosts() {  
  const response = await fetch(  
    process.env.REACT_APP_API_URL!  
  );  
  const body = await response.json()  
  return body;  
}
```

Using the effect hook with fetch

Strongly typing response data

- We will use the **unknown type** with a slightly different TypeScript feature called a **type assertion function** to type the response data in the **getPosts** function.
- Carry out the following steps:
 - Add a type assertion to the JSON response so that the **body** variable has a type of

```
unknown: export async function getPosts() {  
    const response = await fetch(postsUrl);  
    const body = (await response.json()) as unknown;  
    return body;  
}
```

Using the effect hook with fetch

Strongly typing response data

- Carry out the following steps:
 - Add the following type assertion function beneath **getPosts**:

```
export function assertIsPosts(  
  postData: unknown  
): asserts postData is PostData[] {  
}
```

- **Notice** the return type annotation: **asserts postData is PostData[]**. This is called an **assertion signature** and specifies that the postData parameter is of the PostData[] type if no error occurs in the function execution.

Using the effect hook with fetch

Strongly typing response data

- Carry out the following steps:
 - Implement of **assertIsPosts**. It will be a series of checks on the **postsData** parameter and it will throw an exception if a check fails. Start the implementation by checking that **postsData** is an array:

```
export function assertIsPosts(  
  postsData: unknown  
): asserts postsData is PostData[] {  
  if (!Array.isArray(postsData)) {  
    throw new Error("posts isn't an array");  
  }  
  if (postsData.length === 0) {  
    return;  
  }  
}
```

Using the effect hook with fetch

Strongly typing response data

- Carry out the following steps:
 - Check on the array items to see whether they have an **id** property:

```
export function assertIsPosts(  
  postData: unknown  
): asserts postData is PostData[] {  
  ...  
  postData.forEach((post) => {  
    if (!('id' in post)) {  
      throw new Error("post doesn't contain id");  
    }  
    if (typeof post.id !== 'number') {  
      throw new Error('id is not a number');  
    }  
  });  
}
```


Using the effect hook with fetch

Strongly typing response data

- Carry out the following steps:
 - Do similar checks for the **title** and **description** properties:

```
postsData.forEach((post) => {  
  // ...  
  if (!('title' in post)) {  
    throw new Error("post doesn't contain title");  
  }  
  if (typeof post.title !== 'string') {  
    throw new Error('title is not a string');  
  }  
  if (!('description' in post)) {  
    throw new Error("post doesn't contain description");  
  }  
  if (typeof post.description !== 'string') {  
    throw new Error('description is not a string');  
  }  
});
```

Using the effect hook with fetch

Strongly typing response data

- Carry out the following steps:
 - Moving back to **getPosts**, add a call to the **assert** function:

```
export async function getPosts() {  
  const response = await fetch(postsUrl);  
  const body = (await response.json()) as unknown;  
  assertIsPosts(body);  
  return body;  
}
```

Using the effect hook with fetch

Strongly typing response data

- Carry out the following steps:
 - Add an import **PostData** statement at the top of **getPosts.ts**
- Add a file called **types.ts** in the **posts** folder with the following definition for the

PostData type:

```
export type PostData = {  
  id: number;  
  title: string;  
  description: string;  
};
```

Using the effect hook with fetch

Creating a blog posts list component

- Carry out the following steps:
 - Create a file called **PostsList.tsx** in the posts folder:

```
import { postData } from './types';
```

- Implement the component:

```
type Props = {  
  posts: postData[];  
};  
  
export function PostsList({ posts }: Props) {  
  
}
```

Using the effect hook with fetch

Creating a blog posts list component

- Carry out the following steps:
 - Render the blog posts in an unordered list:

```
export function PostsList({ posts }: Props) {  
  return (  
    <ul className="list-none">  
      {posts.map((post) => (  
        <li key={post.id} className="border-b py-4">  
          <h3 className="text-slate-900 font-bold">  
            {post.title}  
          </h3>  
          <p className="text-slate-900">{post.  
            description}</p>  
        </li>  
      ))}  
    </ul>  
  );  
}
```

Using the effect hook with fetch

Creating a blog posts page component

- Carry out the following steps:
 - Create a file called **PostsPage.tsx** in the **posts** folder components with the following import statements:

```
import { useEffect, useState } from 'react';  
import { getPosts } from '../getPosts';  
import { postData } from '../types';  
import { PostsList } from '../PostsList';
```

Using the effect hook with fetch

Creating a blog posts page component

- Carry out the following steps:
 - Define the state for the blog posts and whether they are being fetched:

```
export function PostsPage() {  
  const [isLoading, setIsLoading] = useState(true);  
  const [posts, setPosts] = useState<PostData[]>([]);  
}
```

Using the effect hook with fetch

Creating a blog posts page component

- Carry out the following steps:
 - Call the **getPosts** function using the **useEffect** hook:

```
export function PostsPage() {  
  ...  
  useEffect(() => {  
    let cancel = false;  
    getPosts().then((data) => {  
      if (!cancel) {  
        setPosts(data);  
        setIsLoading(false);  
      }  
    });  
    return () => {  
      cancel = true;  
    };  
  }, []);  
}
```


Using the effect hook with fetch

Creating a blog posts page component

- Carry out the following steps:
 - Add a **loading** indicator while the data is being fetched after the call to **useEffect**:

```
export function PostsPage() {  
  ...  
  useEffect(...);  
  if (isLoading) {  
    return (  
      <div className="w-96 mx-auto mt-6">  
        Loading ...  
      </div>  
    );  
  }  
}
```

Using the effect hook with fetch

Creating a blog posts page component

- Carry out the following steps:
 - Render a page **title** and the **posts list** after the conditional **loading** indicator:

```
export function PostsPage() {  
  ...  
  if (isLoading) {  
    return (  
      <div className="w-96 mx-auto mt-6">  
        Loading ...  
      </div>  
    );  
  }  
  return (  
    <div className="w-96 mx-auto mt-6">  
      <h2 className="text-xl text-slate-900 font-  
bold">Posts</h2>  
      <PostsList posts={posts} />  
    </div>  
  );  
}
```

Using the effect hook with fetch

Creating a blog posts page component

- Carry out the following steps:
 - Open **App.tsx** and replace its contents with the following so that it renders the page component we just created:

```
import { PostsPage } from '../posts/PostsPage';

function App() {
  return <PostsPage />;
}

export default App;
```

- Run the app in a new terminal separate from the one running the REST API.

Using the effect hook with fetch

Here are the key points on interacting with HTTP GET requests in a REST API using **fetch** and **useEffect**:

- **fetch** will make the actual HTTP request that has the REST API's URL as a parameter
- A **type assertion** function can be used to **strongly type response data**
- **useEffect** can trigger the **fetch** call when the component that holds the data in state is mounted
- A flag can be used inside **useEffect** to check that the component hasn't been unmounted during the HTTP request before the data state is set

Posting data with fetch

- **Description:** we will create a form that submits a new blog post to our REST API, and use **fetch** to post to the REST API. That function will be called in the form's submit handler.



Posting data with fetch

Creating new blog posts using fetch

- Carry out the following steps:
 - Open **types.ts** in the **posts** folder and adding the following two types:

```
export type NewPostData = {  
  title: string;  
  description: string;  
};
```

```
export type SavedPostData = {  
  id: number;  
};
```

Posting data with fetch

Creating new blog posts using fetch

- Carry out the following steps:
 - Create a new file called **savePost.ts** in the **posts** folder and add the following import statement:

```
import { NewPostData, SavedPostData } from './types';
```

Posting data with fetch

Creating new blog posts using fetch

- Carry out the following steps:
 - Implement the **savePost** function:

```
export async function savePost(  
  postData: NewPostData  
) {  
  const response = await fetch(  
    process.env.REACT_APP_API_URL!,  
    {  
      method: 'POST',  
      body: JSON.stringify(newPostData),  
      headers: {  
        'Content-Type': 'application/json',  
      },  
    }  
  );  
}
```


Posting data with fetch

Creating new blog posts using fetch

- Carry out the following steps:
 - Strongly type the response body:

```
export async function savePost(newPostData: NewPostData)
{
  const response = await fetch( ... );
  const body = (await response.json()) as unknown;
  assertIsSavedPost(body);
}
```

Posting data with fetch

Creating new blog posts using fetch

- Carry out the following steps:
 - Finish the implementation of **savePost** by merging the blog post ID from the response with the blog post title and description supplied to the function:

```
export async function savePost(newPostData: NewPostData)
{
  ...
  return { ...newPostData, ...body };
}
```

Posting data with fetch

Creating new blog posts using fetch

- Carry out the following steps:
 - The last step is to implement the type assertion function:

```
function assertIsSavedPost(  
  post: any  
): asserts post is SavedPostData {  
  if (!('id' in post)) {  
    throw new Error("post doesn't contain id");  
  }  
  if (typeof post.id !== 'number') {  
    throw new Error('id is not a number');  
  }  
}
```

Posting data with fetch

Creating a blog post form component

▪ Description:

- We will create a component that contains a form that captures a new blog post. When the form is submitted, it will call the **savePost** function we just created.
- We will use **React Hook Form** to implement the form along with a **ValidationError** component.

Posting data with fetch

Creating a blog post form component

- Carry out the following steps:

- Create a file called

ValidationError.tsx in the **posts**

folder

```
import { FieldError } from 'react-hook-form';

type Props = {
  fieldError: FieldError | undefined,
};

export function ValidationError({ fieldError }: Props) {
  if (!fieldError) {
    return null;
  }
  return (
    <div role="alert" className="text-red-500 text-xs mt-1">
      {fieldError.message}
    </div>
  );
}
```

Posting data with fetch

Creating a blog post form component

- Carry out the following steps:
 - Create a new file in the **posts** folder called **NewPostForm.tsx**

```
import { FieldError, useForm } from 'react-hook-form';  
import { ValidationError } from '../ValidationError';  
import { NewPostData } from '../types';
```

- Implement the form component

```
type Props = {  
  onSave: (newPost: NewPostData) => void;  
};  
  
export function NewPostForm({ onSave }: Props) {  
}
```

Posting data with fetch

Creating a blog post form component

- Carry out the following steps:
 - Destructure the **register** and **handleSubmit** functions and useful state variables from React Hook Form's **useForm** hook:

```
type Props = {
  onSave: (newPost: NewPostData) => void;
};

export function NewPostForm({ onSave }: Props) {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting, isSubmitSuccessful
  },
  } = useForm<NewPostData>();
}
```

Posting data with fetch

Creating a blog post form component

- Carry out the following steps:
 - Create a variable for the field container style and a function for the editor style:

```
export function NewPostForm({ onSave }: Props) {  
  ...  
  const fieldStyle = 'flex flex-col mb-2';  
  function getEditorStyle(  
    fieldError: FieldError | undefined  
  ) {  
    return fieldError ? 'border-red-500' : '';  
  }  
}
```


Posting data with fetch

Creating a blog post form component

- Carry out the following steps:
 - Render title and description fields in a form element:

```
export function NewPostForm({ onSave }: Props) {  
  ...  
  return (  
    <form  
      noValidate  
      className="border-b py-4"  
      onSubmit={handleSubmit(onSave)}  
    >  
      <div className={fieldStyle}>  
        <label htmlFor="title">Title</label>  
        <input  
          type="text"  
          id="title"  
          {...register('title', {  
            required: 'You must enter a title',  
          })}  
          className={getEditorStyle(errors.title)}  
        />  
        <ValidationError fieldError={errors.title} />  
      </div>  
      <div className={fieldStyle}>  
        <label htmlFor="description">Description</label>  
        <textarea  
          id="description"  
          {...register('description', {  
            required: 'You must enter the description',  
          })}  
          className={getEditorStyle(errors.description)}  
        />  
        <ValidationError fieldError={errors.description} />  
      </div>  
    </form>  
  );  
}
```

Posting data with fetch

Creating a blog post form component

- Carry out the following steps:
 - Render a **Save** button and the success message:

```
<form
  noValidate
  className="border-b py-4"
  onSubmit={handleSubmit(onSave)}
>

<div className={fieldStyle}> ... </div>
<div className={fieldStyle}> ... </div>
<div className={fieldStyle}>
  <button
    type="submit"
    disabled={isSubmitting}
    className="mt-2 h-10 px-6 font-semibold bg-black
      text-white"
  >
    Save
  </button>
  {isSubmitSuccessful && (
    <div
      role="alert"
      className="text-green-500 text-xs mt-1"
    >
      The post was successfully saved
    </div>
  )}
</div>
</form>
```

Posting data with fetch

Creating a blog post form component

- Carry out the following steps:
 - Open **PostPage.tsx** and import the **NewPostForm** component, the **savePost** function, and the **NewPostData** type:

```
import { useEffect, useState } from 'react';
import { getPosts } from '../getPosts';
import { postData, NewPostData } from '../types';
import { PostsList } from '../PostsList';
import { savePost } from '../savePost';
import { NewPostForm } from '../NewPostForm';
```

Posting data with fetch

Creating a blog post form component

- Carry out the following steps:
 - In the **PostPage** JSX, add the **NewPostForm** form above the **PostsList**:

```
<div className="w-96 mx-auto mt-6">  
  <h2 className="text-xl text-slate-900 font-  
    bold">Posts</h2>  
  <NewPostForm onSave={handleSave} />  
  <PostsList posts={posts} />  
</div>;
```

Posting data with fetch

Creating a blog post form component

- Carry out the following steps:
 - Add the save handler function just below the effect that gets blog posts:

```
useEffect(() => {  
    ...  
}, []);  
  
async function handleSave(newPostData: NewPostData) {  
    const newPost = await savePost(newPostData);  
    setPosts([newPost, ...posts]);  
}
```

Posting data with fetch

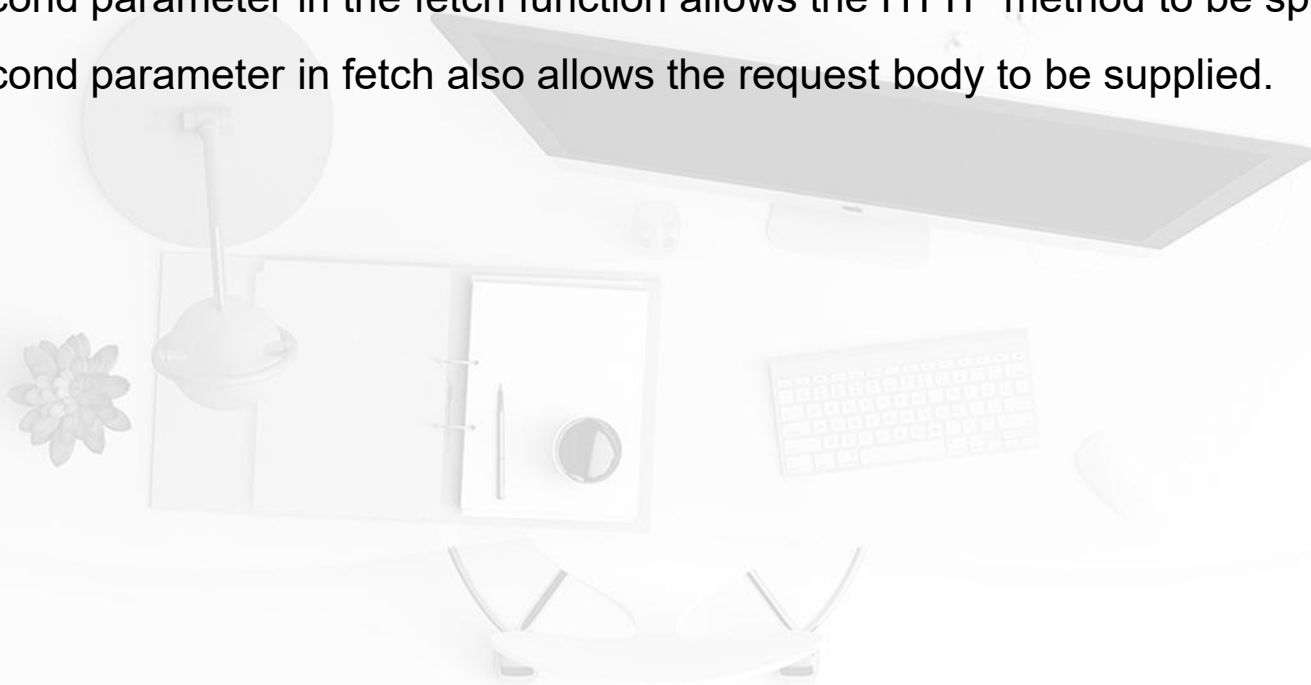
Creating a blog post form component

- Carry out the following steps:
 - The handler calls **savePost** with the data from the form. After the post has been saved, it is added to the start of the posts array.
 - In the running app, the new blog post form will appear above the blog post list.
 - Fill in the form with a new blog post and press the **Save** button. The new post should appear at the top of the list after a couple of seconds.

Posting data with fetch

Recap

- The second parameter in the fetch function allows the HTTP method to be specified.
- The second parameter in fetch also allows the request body to be supplied.



Updating data with fetch

To update an existing item, follow this workflow:

- **Fetch:** Retrieve the specific resource data by its id.
- **Populate:** Pass the fetched data into your form (pre-filling).
- **Submit:** Use PUT (or PATCH) to send updated data to the API.

Updating data with fetch

Example

Fetching Data by ID.

```
export async function fetchPostById(id: string) {  
  const response = await fetch(`${import.meta.env.VITE_API_URL}/${id}`);  
  if (!response.ok) throw new Error('Could not fetch post');  
  return response.json();  
}
```

Updating data with fetch

Example

When the form is submitted, use the PUT method to update the resource.

```
export async function updatePost(id: string, data: PostData) {  
  const response = await fetch(`${import.meta.env.VITE_API_URL}/${id}`, {  
    method: 'PUT',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify(data),  
  });  
  return response.json();  
}
```



Deleting data with fetch

Deleting a resource requires caution.

- **Trigger:** Click event on a “Delete” button.
- **Confirm:** Always ask for user confirmation.
- **Execute:** Send the DELETE request to the API.

Deleting data with fetch

Use `window.confirm()` for a simple native browser confirmation.

```
const handleDelete = (id: string) => {  
  const confirmed = window.confirm("Are you sure you want to delete this?");  
  if (confirmed) {  
    executeDelete(id);  
  }  
};
```

Deleting data with fetch

Executing DELETE Request.

```
export async function deletePost(id: string) {  
  const response = await fetch(`${import.meta.env.VITE_API_URL}/${id}`, {  
    method: 'DELETE',  
  });  
  
  if (!response.ok) throw new Error('Delete operation failed');  
  return id; // Return ID for local state cleanup  
}
```

Using React Router

Understanding React Router data loading

- React Router data loading is similar to React Router forms.
- Instead of defining an action that handles form submission, we define a **loader** that handles data loading.
- The following code snippet defines a loader on a some-page route:

```
const router = createBrowserRouter([
  ...,
  {
    path: '/some-page',
    element: <SomePage />,
    loader: async () => {
      const response = fetch('https://somewhere');
      return await response.json();
    }
  },
  ...
]);
```

Using React Router

Understanding React Router data loading

- React Router calls the loader to **get the data before** it **renders** the component defined on the route.
- The data is then available in the component via a **useLoaderData** hook:

```
export function SomePage() {  
  const data = useLoaderData();  
  ...  
}
```

Using React Router

Using React Router for data loading

- Carry out the following steps:
 - Open **App.tsx** and add the following import statement:

```
import {  
  createBrowserRouter,  
  RouterProvider  
} from 'react-router-dom';
```

- Import the **getPosts** function:

```
import { getPosts } from './posts/getPosts';
```


Using React Router

Using React Router for data loading

- Carry out the following steps:
 - Add the following router definition above the **App** component:

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <PostsPage />,
    loader: getPosts
  }
]);
```

Using React Router

Using React Router for data loading

- Carry out the following steps:
 - In the **App** component, replace **PostsPage** with **RouterProvider**:

```
function App() {  
  return <RouterProvider router={router} />;  
}
```

- Open **PostsPage.tsx** and remove the React import statement, as this is no longer required in this component.
- Add **assertIsPosts** to the **getPosts** import statement and remove **getPosts**:

```
import { assertIsPosts } from './getPosts';
```

Using React Router

Using React Router for data loading

- Carry out the following steps:
 - Still in **PostsPage.tsx**, add the following import statement for a hook in React Router that allows us to access the loader data:

```
import { useLoaderData } from 'react-router-dom';
```
 - Inside the **PostsPage** component, remove the **isLoading** and **posts** state definitions. These won't be needed because we will get the data from React Router without having to do any waiting.
 - Remove the call to **useEffect** that currently gets the data.

Using React Router

Using React Router for data loading

- Carry out the following steps:
 - Remove the second line of the **handleSave** function that sets the **posts** state.

handleSave should now read:

```
async function handleSave(newPostData: NewPostData) {  
  await savePost(newPostData);  
}
```

- Remove the loading indicator.

Using React Router

Using React Router for data loading

- Carry out the following steps:
 - Now at the top of the **PostsPage** component, make a call to **useLoaderData** and assign the result to a **posts** variable:

```
export function PostsPage() {  
  const posts = useLoaderData();  
  ...  
}
```

Using React Router

Using React Router for data loading

- Carry out the following steps:
 - **posts** is of the **unknown** type, so there is a type error where it is passed to the **PostsLists** component. Use the **assertIsPosts** function to type the data with **PostData[]**:

```
const posts = useLoaderData();  
assertIsPosts(posts);
```

Using React Router

Deferred React Router data fetching

- If the **data-fetching process is slow**, there will be a **noticeable delay** before a component is rendered by React Router.
- There is a solution to this using React Router's **defer** function and **Await** component, along with React's **Suspense** component.

Using React Router

Deferred React Router data fetching

- Carry out the following steps:
 - Open **App.tsx** and add the **defer** function to the React Router import statement:

```
import {  
  createBrowserRouter,  
  RouterProvider,  
  defer  
} from 'react-router-dom';
```


Using React Router

Deferred React Router data fetching

- Carry out the following steps:
 - Update the **loader** function as follows in the route definition:

```
const router = createBrowserRouter([
  {
    path: "/",
    element: ...,
    loader: async () => defer({ posts: getPosts() })
  }
]);
```

Using React Router

Deferred React Router data fetching

- Carry out the following steps:
 - Open **PostsPage.tsx** and add an import statement for React's **Suspense** component:

```
import { Suspense } from 'react';
```

- Add the **Await** component to the React Router import statement:

```
import { useLoaderData, Await } from 'react-router-dom';
```

- In the component, update the call to **useLoaderData** to assign the result to a data variable instead of **posts**:

```
const data = useLoaderData()
```

- Remove the call to **assertIsPosts**.

Using React Router

Deferred React Router data fetching

- Carry out the following steps:
 - The data variable is of the **unknown** type, so add a type assertion function beneath the component that can be used to strongly type it:

```
type Data = {  
  posts: PostData[];  
};  
  
export function assertIsData(  
  data: unknown  
): asserts data is Data {  
  if (typeof data !== 'object') {  
    throw new Error("Data isn't an object");  
  }  
  if (data === null) {  
    throw new Error('Data is null');  
  }  
  if (!('posts' in data)) {  
    throw new Error("data doesn't contain posts");  
  }  
}
```

Using React Router

Deferred React Router data fetching

- Carry out the following steps:
 - Use the assertion function to type the **data** variable in the component:

```
const data = useLoaderData();  
assertIsData(data);
```

Using React Router

Deferred React Router data fetching

- Carry out the following steps:
 - In the JSX, wrap **Suspense** and **Await** around **PostsList**

```
<Suspense fallback={<div>Fetching...</div>}>  
  <Await resolve={data.posts}>  
    {(posts) => {  
      assertIsPosts(posts);  
      return <PostsList posts={posts} />;  
    }}  
  </Await>  
</Suspense>
```

- In the running app, you will now notice the Fetching... message when the page loads:

Using React Router

Recap

- React Router's **loader** allows us to efficiently load fetched data into a route component
- React Router's **defer** allows the route component not to be blocked from rendering the component while data is being fetched
- React Router's **useLoaderData** hook allows a component to access a route's loader data
- React's **Suspense** and React Router's **Await** allow a component to render while data is still being fetched

Summary

- We used the browser's fetch function to make HTTP **GET** and **POST** requests. The request's **URL** is the **first argument** on the fetch function. The **second argument** on fetch allows the **request options** to be specified, such as the HTTP method and body.
- A **type assertion function** can be used to strongly type the data in the response body of an HTTP request. The function takes in the data having an **unknown** type. The function then carries out checks to validate the type of data and throws an error if it is invalid. If no errors occur, the asserted type for the data is specified in the functions assertion signature.

Summary

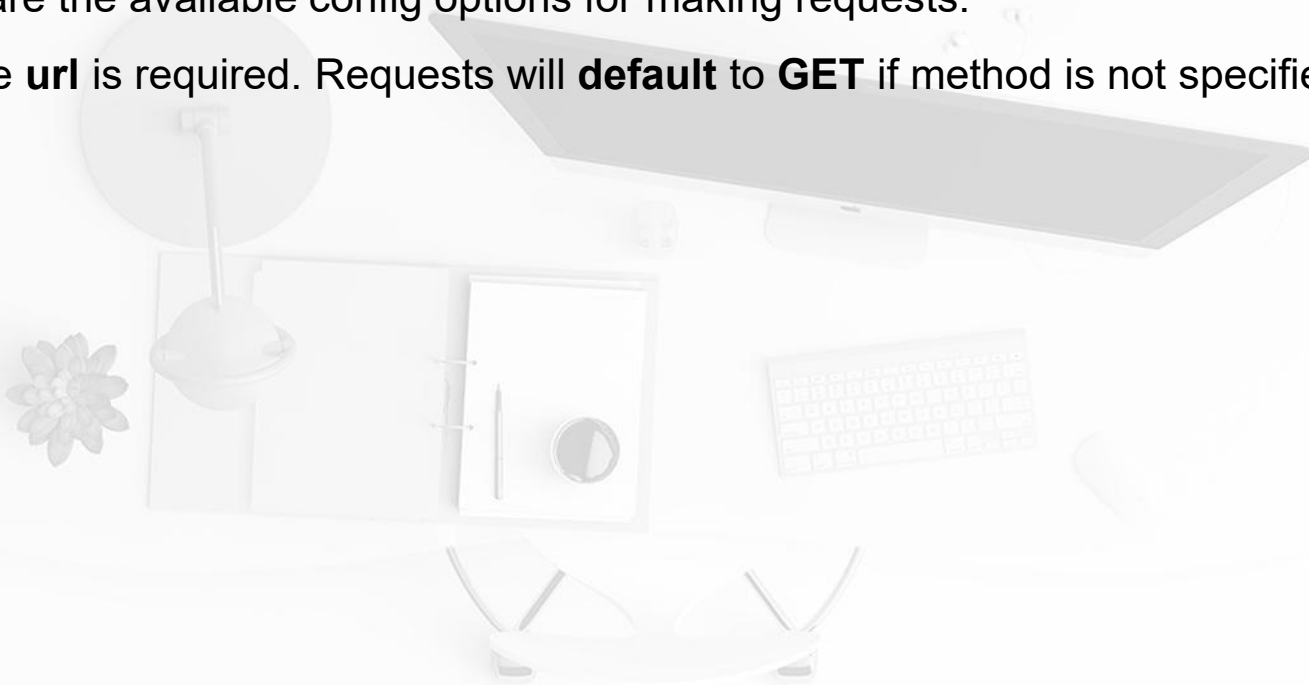
- React's **useEffect** hook can be used to execute a call to fetch data from a backend API and store the data in the state when the component is mounted. A flag can be used inside **useEffect** to ensure the component is still mounted after the HTTP request before the data state is set.

AXIOS

- Axios is a promise-based HTTP Client for node.js and the browser.
- Features:
 - Make XMLHttpRequests from the browser
 - Make http requests from node.js
 - Supports the Promise API
 - Automatic request body serialization to: JSON, Multipart / FormData, URL encoded form
 - Posting HTML forms as JSON
 - Automatic JSON data handling in response
 - ...
- **Installing:** npm install axios

AXIOS - Request Config

- These are the available config options for making requests.
- Only the **url** is required. Requests will **default** to **GET** if method is not specified.



AXIOS - Request Config

Common options

```
{  
  // `url` is the server URL that will be used for the request  
  url: '/user',  
  
  // `method` is the request method to be used when making the request  
  method: 'get', // default  
  
  // `params` are the URL parameters to be sent with the request  
  // Must be a plain object or a URLSearchParams object  
  params: {  
    ID: 12345  
  },  
  
  // `data` is the data to be sent as the request body  
  // Only applicable for request methods 'PUT', 'POST', 'DELETE', and 'PATCH'  
  data: {  
    firstName: 'Fred'  
  },  
  
  // `responseType` indicates the type of data that the server will respond  
  responseType: 'json', // default  
  ...  
}
```

AXIOS - Response Schema

Common information

- The response for a request contains the following information.

```
{  
  // `data` is the response that was provided by the server  
  data: {},  
  
  // `status` is the HTTP status code from the server response  
  status: 200,  
  
  // `statusText` is the HTTP status message from the server response  
  statusText: 'OK',  
  ...  
}
```

AXIOS - Performing a GET request

<https://www.npmjs.com/package/axios>

```
import axios from 'axios';  
  
//const axios = require('axios'); // legacy way  
  
// Make a request for a user with a given ID  
axios.get('/user?ID=12345')  
  .then(function (response) {  
    // handle success  
    console.log(response);  
  })  
  .catch(function (error) {  
    // handle error  
    console.log(error);  
  })  
  .finally(function () {  
    // always executed  
  });
```

```
// Want to use async/await?  
async function getUser() {  
  try {  
    const response = await axios.get('/user?ID=12345');  
    console.log(response);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

AXIOS - Performing a POST request

```
axios.post('/user', {  
  firstName: 'Fred',  
  lastName: 'Flintstone'  
})  
  .then(function (response) {  
    console.log(response);  
  })  
  .catch(function (error) {  
    console.log(error);  
  });
```

AXIOS

Performing multiple concurrent requests

```
function getUserAccount() {  
  return axios.get('/user/12345');  
}  
  
function getUserPermissions() {  
  return axios.get('/user/12345/permissions');  
}  
  
Promise.all([getUserAccount(), getUserPermissions()])  
  .then(function (results) {  
    const acct = results[0];  
    const perm = results[1];  
  });
```

AXIOS API

```
// Send a POST request
axios({
  method: 'post',
  url: '/user/12345',
  data: {
    firstName: 'Fred',
    lastName: 'Flintstone'
  }
});
```


Request method aliases

- For convenience, aliases have been provided for all common request methods.

- `axios.get(url[, config])`
- `axios.delete(url[, config])`
- `axios.post(url[, data[, config]])`
- `axios.put(url[, data[, config]])`
- `axios.patch(url[, data[, config]])`

- **NOTE**

When using the alias methods `url`, `method`, and `data` properties don't need to be specified in `config`.



Q&A